Lab 7: Fuzzing for Input Validation Bugs

R Saurav Srrinivas | SRN: **PES1UG24CS352**
PES University, Department of Computer Science and Engineering
Assigned repository: SETAPESU26/L6_L7_43_warranty_claims (Lab 7)

Module under test

`warranty_claims.py` has three functions that read user input: `parse_claim` turns `plan:months` into a (plan, age) pair, `age_average` averages comma-separated product ages, and `payable_charge` adds 18% GST to a repair charge and subtracts a discount from `charge|discount`. None of them validated their input.

Step 1: Install dependencies

`pip install hypothesis` and `pip install pytest` (Python 3.13.18, pytest 9.1.1, Hypothesis 6.170.15).

Step 2: Run the buggy application

With the sample inputs the script works. With an input that has no colon it crashes straight away with an unhandled `ValueError`.

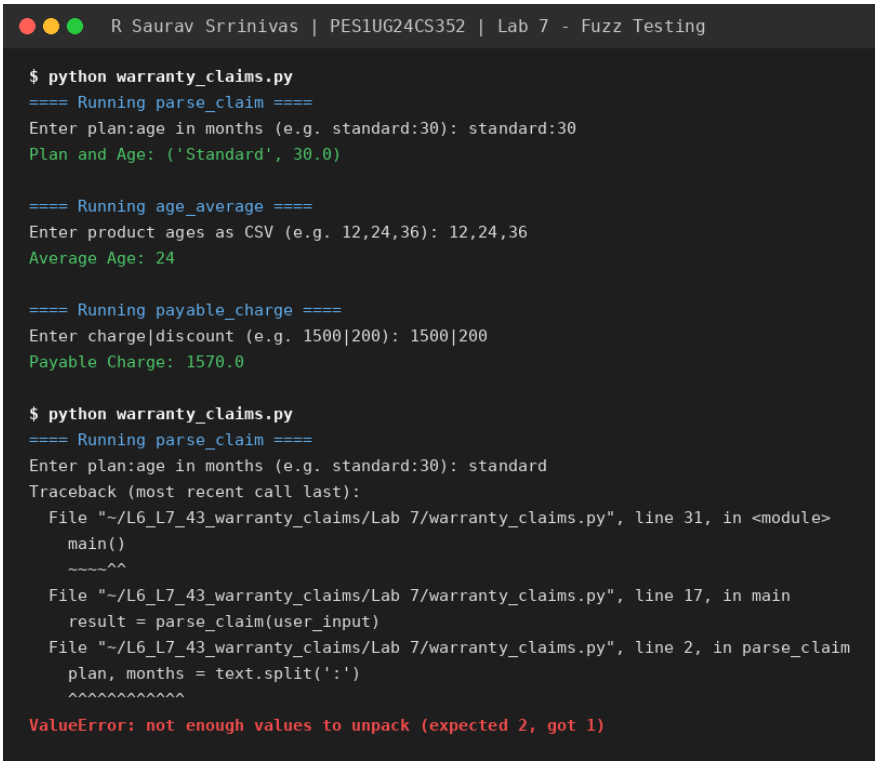


Figure 1: python warranty_claims.py on the original code: sample input works, 'standard' crashes

Step 3: Hypothesis tests written

`test_warranty_claims.py` has nine tests, three per function, each run on 300 generated inputs:

Test kind	Strategy	Property checked
No crash (the starter tests)	`st.text() | st.none()`	The function never raises, whatever it is given.

Test kind	Strategy	Property checked
Near-valid structured input	Real separator with odd values: nan, inf, -1, 1e400, empty, junk; lists of negative/positive integers	No crash, and anything returned is sensible: non-empty plan, finite non-negative age or charge, no negative discount, average between min and max.
Valid input	Lowercase plan + 0..600 months; 1-20 ages in 0..1200; charge and discount in 0..1e6	The result matches the expected value, so the fix did not break normal behaviour.

Step 4: Run the tests on the buggy code

7 of the 9 tests fail. Only the two valid-input tests for `parse_claim` and `payable_charge` pass.

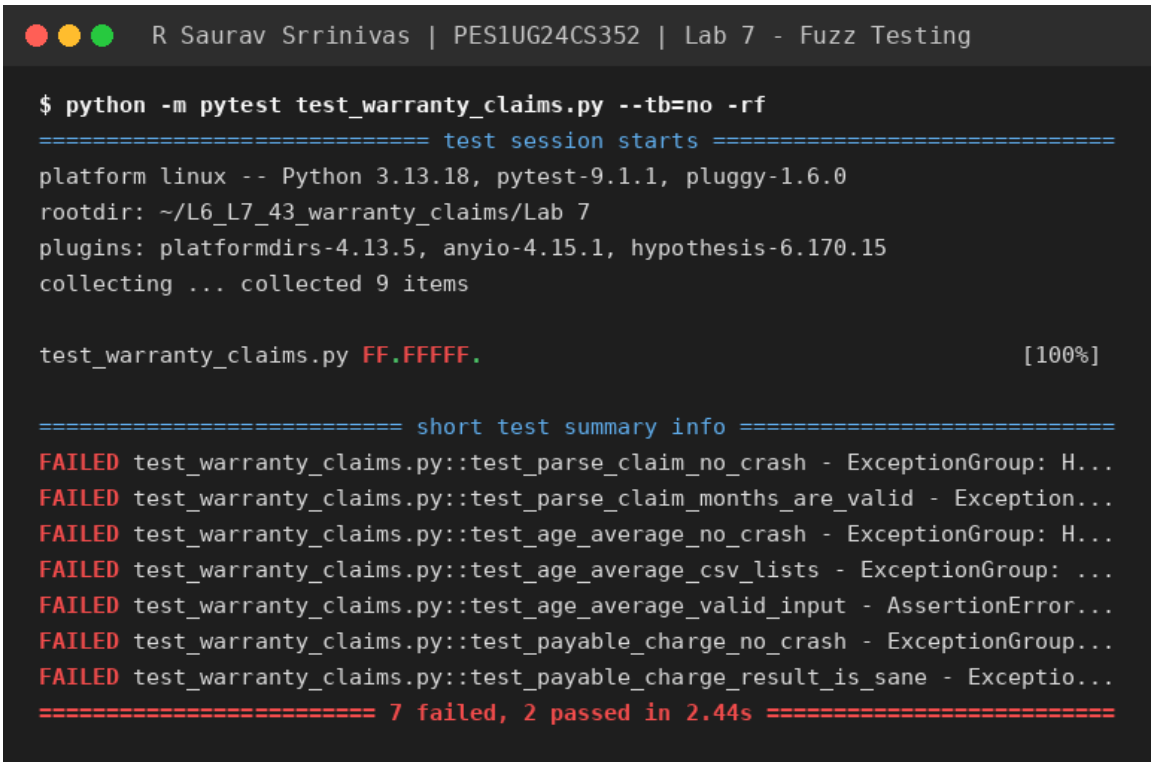


Figure 2: python -m pytest test_warranty_claims.py on the original code: 7 failed, 2 passed

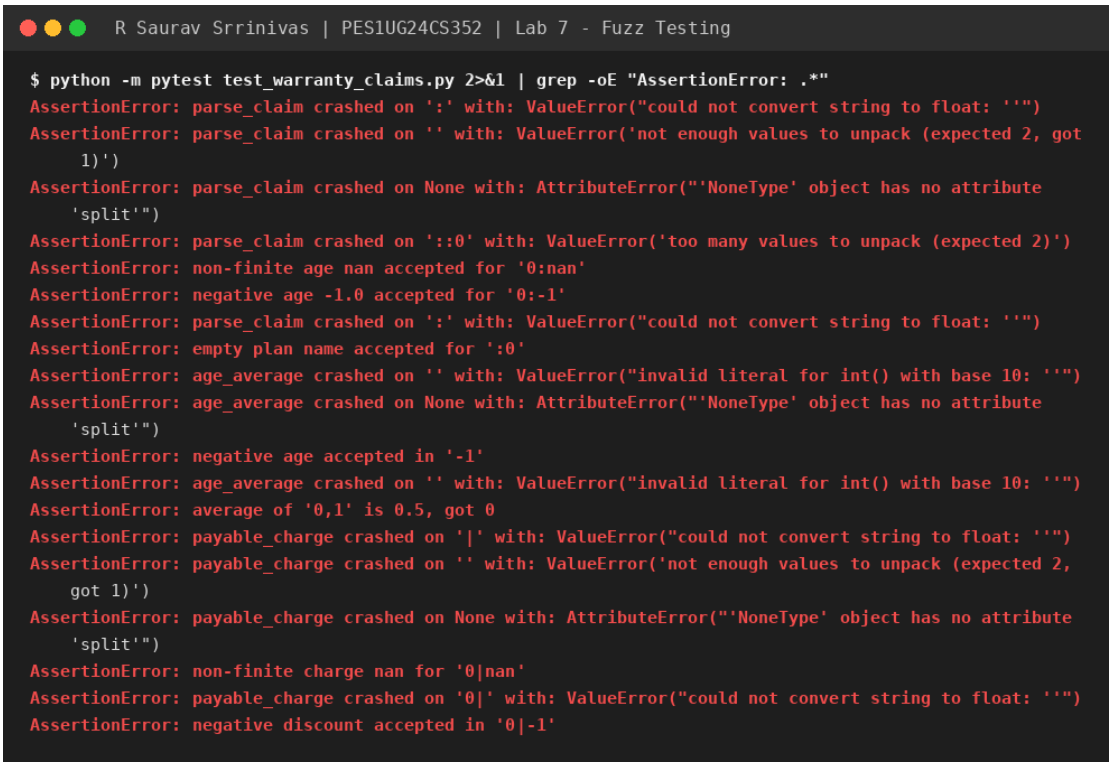


Figure 3: the minimal failing inputs Hypothesis found, taken from the full pytest output

Bugs found and how they were fixed

Each function now returns `None` for input it cannot use instead of raising an exception, so the no-crash tests can hold for every input.

Function	Failing input found	Problem	Fix
all three	`None`	AttributeError on .split()	Return None if input is not a str.
all three	`''`, `'::0'`, `'|'`	Unpacking a split with the wrong number of parts raises ValueError	Check the split gives exactly 2 parts.
parse_claim	`':'`, `':0'`	Empty month crashes float(); empty plan name accepted	Plan must have a letter/digit and only letters, digits, space, - or _.
parse_claim	`'0:nan'`, `'0:-1'`	nan, inf and negative months accepted	Months must be finite and >= 0.
age_average	`''`	int('') raises; empty list would divide by zero	Reject blank input and any token that is not a whole number.
age_average	`'-1'`	Negative ages accepted	Reject ages below 0.
age_average	`'0,1'`	Floor division gives 0 instead of 0.5	True division rounded to 2 places; OverflowError caught.
payable_charge	`'0|'`	Empty number crashes float()	Parse each side safely.
payable_charge	`'0|nan'`	max(nan, 0) returns nan, so nan is shown as the charge	Charge and discount must be finite and >= 0; the total must be finite.
payable_charge	`'0|-1'`	Negative discount raises the bill	Reject negative discount; result rounded to 2 places.
main	a bad line typed by the user	One bad value kills the whole program	Print an 'Invalid input' message and carry on; EOF on input is handled.

Step 5: Run the fixed application

Sample input gives the same results as before, except the average is now an exact value (24.0). Bad input gets a clear message instead of a traceback.

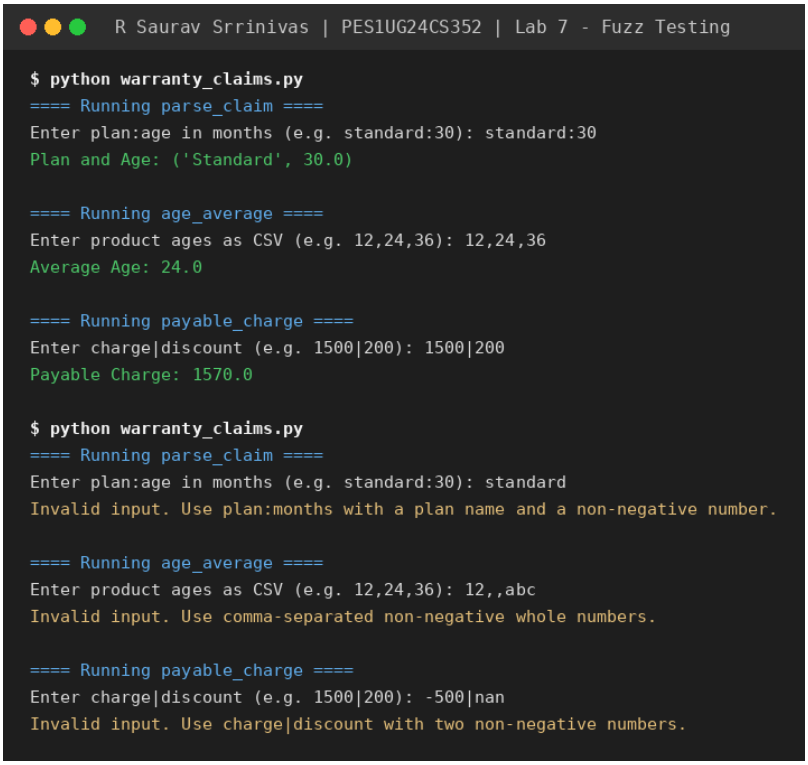


Figure 4: python warranty_claims.py on the fixed code: valid and invalid inputs

Step 6: Re-run the tests on the fixed code

All 9 tests pass. As an extra check the suite was also run with 5,000 examples per test, and everything still passed.

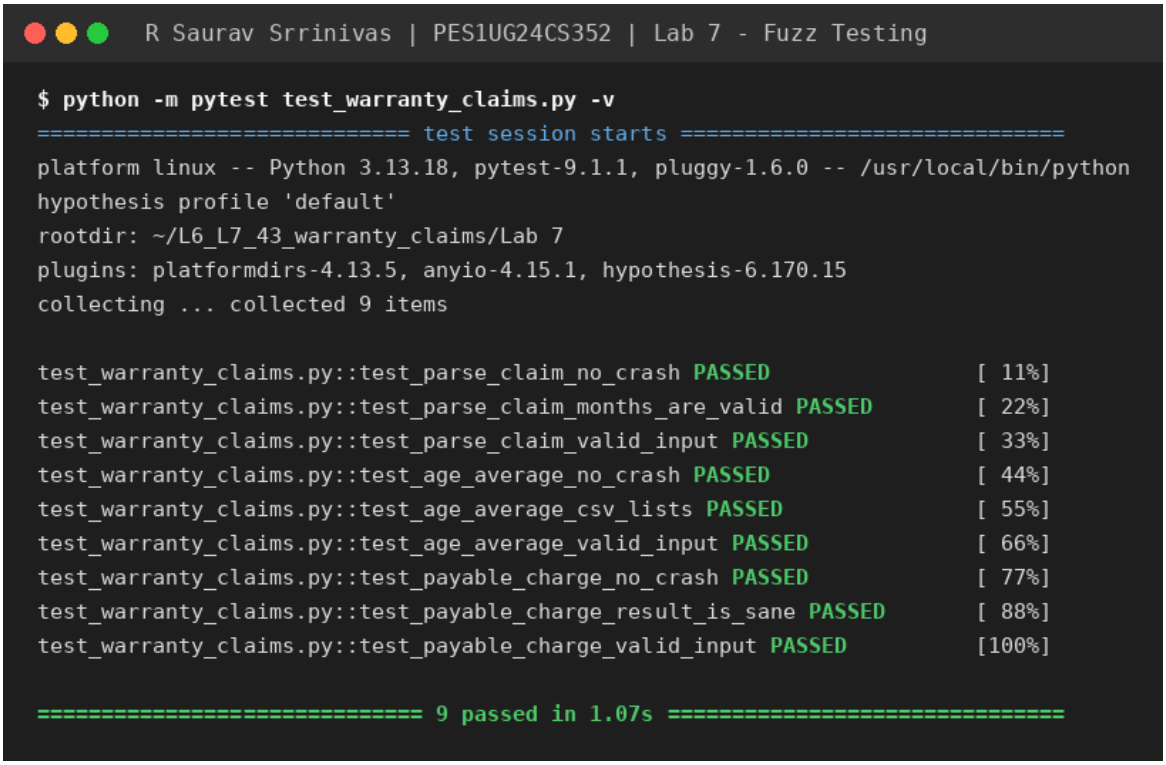


Figure 5: python -m pytest test_warranty_claims.py -v on the fixed code: 9 passed

Deliverables

#	Deliverable	File
1	PDF document with the screenshots	`PES1UG24CS352_Lab7_FuzzTesting.pdf`
2	Fixed application Python script	`warranty_claims.py`
3	Updated Hypothesis test file	`test_warranty_claims.py`